



Objetos

Marcar como feito

Objetos em JavaScript — Estrutura “chave e valor”

1) O que é um objeto?

Um **objeto** é um conjunto de **pares (chave, valor)**. As chaves são, em geral, *strings* (ou *Symbols*) e cada chave aponta para um valor.

```
const usuario = {  
  nome: "Ana",  
  idade: 28,  
  ativo: true  
};  
// chaves: nome, idade, ativo  
// valores: "Ana", 28, true
```

2) Como declarar (forma literal)

Use chaves `{ }` com pares **chave: valor** separados por vírgula.

```
const produto = {  
  id: 101,  
  titulo: "Caderno",  
  preco: 12.9  
};
```

Chaves com espaços, hífens ou iniciadas por número devem estar **entre aspas**:

```
const exemplo = {  
  "cor-favorita": "azul",  
  "nome completo": "João da Silva",
```

```
"1abc": 42
};
```

3) Acessar valores: ponto vs. colchetes

Notação de ponto (para chaves que são identificadores simples):

```
console.log(produto.titulo); // "Caderno"
```

Notação de colchetes (para chaves dinâmicas ou com caracteres especiais):

```
const campo = "preco";
console.log(produto["preco"]); // 12.9
console.log(produto[campo]); // 12.9 (dinâmico)

const estranho = { "1abc": 42, "cor-favorita": "azul" };
console.log(estranho["1abc"]); // 42
console.log(estranho["cor-favorita"]); // "azul"
```

4) Adicionar e atualizar pares

Atribuir a uma chave **cria** (se não existir) ou **atualiza** (se já existir).

```
const perfil = { nome: "Lia" };

// adicionar
perfil.cidade = "Porto Alegre";
perfil["pais"] = "Brasil";

// atualizar
perfil.nome = "Lia Souza";
```

5) Remover pares

Use o operador **delete** na chave desejada.

```
delete perfil.cidade;
delete perfil["pais"];
```

6) Verificar existência de chave

Use o operador **in** para checar se a **chave** existe no objeto.

```
console.log("nome" in perfil); // true
console.log("cidade" in perfil); // false
```

```
// Diferença importante:  
// obj.chave === undefined verifica o VALOR ser undefined.  
// "chave" in obj verifica a EXISTÊNCIA da chave.
```

7) Chaves calculadas (dinâmicas) na criação

Monte o nome da chave a partir de uma variável ou expressão usando colchetes `[]` no literal.

```
const campoBase = "nota";  
const aluno = {  
  nome: "Bia",  
  [campoBase + "_final"]: 9.2 // cria a chave "nota_final"  
};  
  
console.log(aluno["nota_final"]); // 9.2
```

8) Objetos aninhados (estrutura hierárquica)

Objetos podem conter **outros objetos** e **arrays**, formando estruturas ricas.

```
const pedido = {  
  numero: 555,  
  cliente: {  
    nome: "Carlos",  
    contato: { email: "carlos@mail.com", fone: "51-9999-0000" }  
  },  
  itens: [  
    { sku: "A1", qtd: 2, valor: 10.0 },  
    { sku: "B2", qtd: 1, valor: 25.0 }  
  ]  
};  
  
// acesso:  
console.log(pedido.cliente.contato.email); // "carlos@mail.com"  
console.log(pedido.itens[0].sku);          // "A1"
```

9) Desestruturação (extrair valores por chave)

A desestruturação facilita obter valores do objeto em variáveis com o mesmo nome da chave (ou renomeando).

```
const usuario = { nome: "Ana", cidade: "NH", idade: 28 };  
  
// nomes iguais às chaves:  
const { nome, cidade } = usuario; // nome = "Ana", cidade = "NH"
```

```
// renomear variáveis:  
const { idade: anos } = usuario; // anos = 28  
  
// valor padrão se a chave não existir:  
const { pais = "Brasil" } = usuario; // pais = "Brasil"
```

10) Objeto x Array (quando usar)

- **Objeto**: estrutura de *chave* → *valor*, acesso por nome da chave.
- **Array**: estrutura por *índice* → *valor*, acesso por posição numérica.

Use **objeto** quando precisa *nomear* cada dado. Use **array** quando a *ordem* e as posições são relevantes.

11) Cópia, referência e imutabilidade prática

Objetos são atribuídos por **referência**: duas variáveis podem apontar para o *mesmo* objeto.

```
const a = { x: 1 };  
const b = a; // b referencia o mesmo objeto de a  
b.x = 99;  
console.log(a.x); // 99 (mudou "a" também)
```

Cópia rasa com spread (clona só o nível superior):

```
const original = { x: 1, y: { z: 2 } };  
const copiaRasa = { ...original };  
  
copiaRasa.x = 100; // ok, independente  
copiaRasa.y.z = 200; // altera "por dentro" em ambos (y continua sendo referência)
```



Cópia profunda (para dados puros) com **structuredClone**:

```
const profundo = structuredClone(original);  
// altera profundamente sem refletir no original
```

Última atualização: domingo, 12 out. 2025, 19:05

Atividade anterior

Print do nível mais alto atingido no jogo

Seguir para...

Próxima atividade

Exercícios Objetos

Entre em contato



Siga nossas redes sociais



 Contate o suporte do site

Você acessou como Lucas Matheus Dias Brum (Sair)

Resumo de retenção de dados

Baixar o aplicativo móvel.

Baixar o aplicativo móvel.